Laboratory work 14

Securing Your Application

We will learn how to secure our application in this chapter. This chapter will explain how to implement authentication in our frontend when we are using **JSON Web Token** (**JWT**) authentication in the backend. First, we will switch on security in our backend to enable JWT authentication. Then, we will create a component for the login functionality. Finally, we will modify our CRUD functionalities to send the token in the request's authorization header to the backend, and implement the logout functionality.

In this chapter, we will cover the following topics:

- Securing the backend
- Securing the frontend

Technical requirements

The Spring Boot application that we created in *Chapter 5*, *Securing Your Backend*, is required (`https://github.com/PacktPublishing/Full-Stack-Development-with-Spring-Boot-3andReact-Fourth-Edition/tree/main/Chapter05`), as is the React app that we used in *Chapter 14*, *Styling the Frontend with React MUI* (`https://github.com/PacktPublishing/Full-StackDevelopment-with-Spring-Boot-3and-React-Fourth-Edition/tree/main/Chapter14`).

The following GitHub link for this chapter will also be useful: `https://github.com/PacktPublishing/Full-Stack-Development-with-Spring-Boot-3-and-ReactFourthEdition/tree/main/Chapter16`.

Securing the backend

In *Chapter 13*, we implemented CRUD functionalities in our frontend using an unsecured backend. Now, it is time to switch on security for our backend and go back to the version that we created in *Chapter 5*, *Securing Your Backend*:

1. Open your backend project with the Eclipse IDE and open the `SecurityConfig.java` file in the editor view. We have commented the security out and allowed everyone access to all endpoints. Now, we can remove that line and also remove the comments from the original version. Now, the `filterChain()` method of your `SecurityConfig.java` file should look like the following:

```
@Bean public SecurityFilterChain filterChain(HttpSecurity http)
throws
Exception {   http.csrf((csrf) -> csrf.disable())
.cors(withDefaults())     .sessionManagement((sessionManagement)
->       sessionManagement.sessionCreationPolicy(
SessionCreationPolicy.STATELESS))     .authorizeHttpRequests(
(authorizeHttpRequests)                                    ->
authorizeHttpRequests.requestMatchers(HttpMethod.POST, "/
login").permitAll().anyRequest().authenticated())
    .addFilterBefore(authenticationFilter,
      UsernamePasswordAuthenticationFilter.class)
.exceptionHandling((exceptionHandling) ->
exceptionHandling.authenticationEntryPoint(exceptionHandler));

  return http.build();
}
```

2. Let's test what happens when the backend is secured again. Run the backend by pressing the **Run** button in Eclipse, and check from the console view that the application started correctly. Run the frontend by typing the `npm run dev` command into your terminal, and the browser should be opened to the address `localhost:5173`.
3. You should now see that the list page and the car list are loading. If you open the developer tools and the **Network** tab, you will notice that the response status is **401 Unauthorized**. This is actually what we want because we haven't yet executed authentication in relation to our frontend:

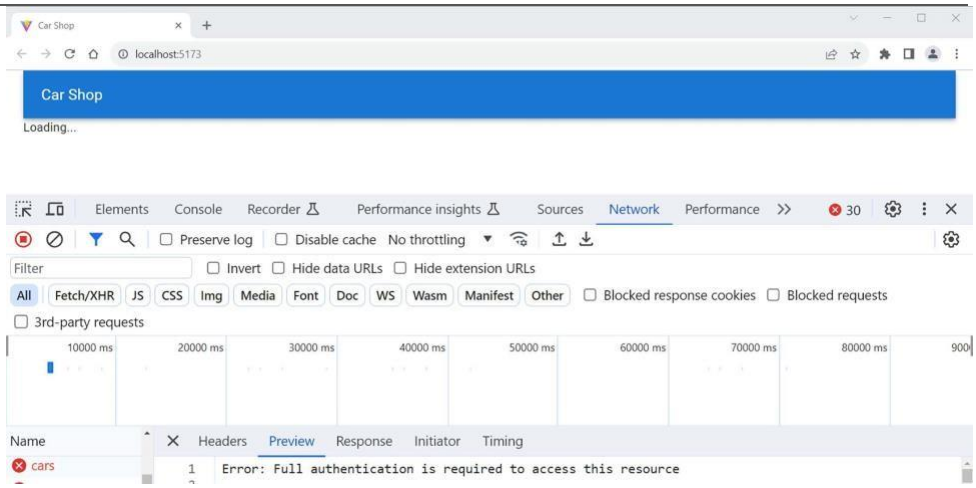

Figure 16.1: 401 Unauthorized

Now, we are ready to work with the frontend.

Securing the frontend

In *Chapter 5*, *Securing Your Backend*, we created JWT authentication and allowed everyone access to the `/login` endpoint without authentication. Now, on the frontend login page, we have to send a `POST` request to the `/login` endpoint using user credentials to get a token. After that, the token will be included in all requests that we send to the backend, as demonstrated in the following figure:

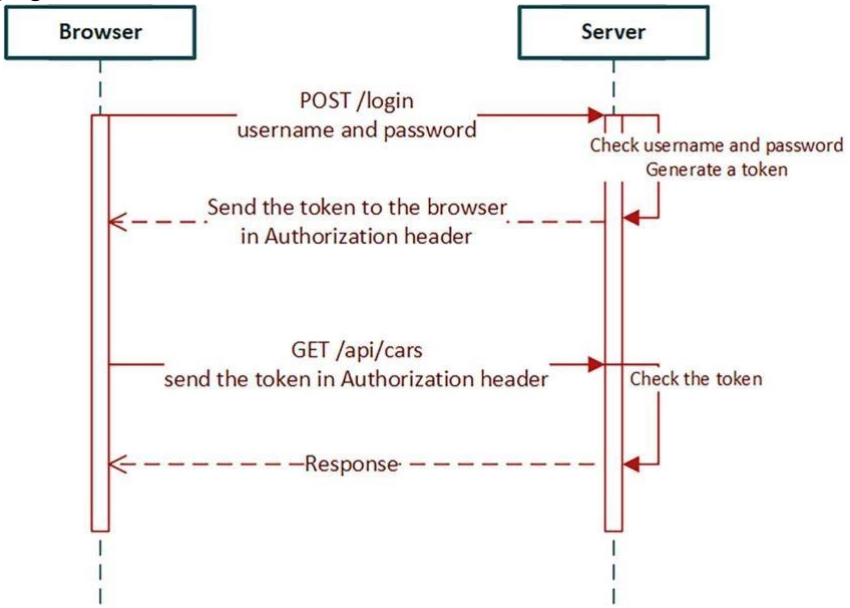

Figure 16.2: Secured application

With this knowledge, we will start to implement login functionality on our frontend. We will implement the login page where the user enters credentials, and then we will send a login request to get a token from the server. We will use the stored token in the requests that we send to the server.

Creating a login component

Let's first create a login component that asks for credentials from the user to get a token from the backend:

1. Create a new file called `Login.tsx` in the `components` folder. Now, the file structure of the frontend should be the following:

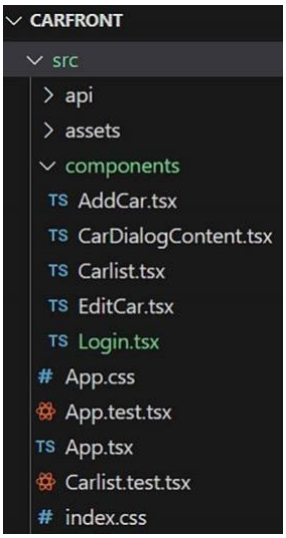

Figure 16.3: Project structure

2. Open the file in the VS Code editor view and add the following base code to the `Login` component. We need `axios` to send `POST` requests to the `/login` endpoint:

```
import { useState } from 'react'; import axios from
'axios';

function Login()
{   return(
<></>
  );
}

export default Login;
```

3. We need two states for the authentication: one for the credentials (username and password), and one boolean value to indicate the status of the authentication. We also create a type for the user state. The initial value of the authentication status state is `false`:

```
import { useState } from 'react'; import
axios from 'axios';
type User =
{
username:
string;
password:
string;
}

function Login() {      const [user, setUser]
= useState<User>({   username: '
                     ,
password: ''
  });   const      [isAuthenticated,
        setAuth] =
useState(false);
    return(
<></>
  );
}

export default Login;
```

4. In the user interface, we are going to use the **Material UI** (**MUI**) component library, as we did with the rest of the user interface. We need `TextField` components for the credentials, the `Button` component to call a login function, and the `Stack` component for layout. Add imports for the components to the `Login.tsx` file:

```
import TextField from '@mui/material/TextField';
import Stack from '@mui/material/Stack';
```

We have already used all three of these component types in *Chapter 14, Styling the Frontend with MUI* to style our UI.

```
import Button from '@mui/material/Button';
```

5. Add the imported components to the `return` statement. We need two `TextField` components: one for the username and one for the password. One `Button` component is

needed to call the login function that we are going to implement later in this section. We use the `Stack` component to align our `TextField` components to the center and to get spacing between them:

```
return(
    <Stack spacing={2} alignItems="center" mt={2}>
      <TextField
name="username"
label="Username"
onChange={handleChange} />
<TextField
type="password"
name="password"
label="Password"
onChange={handleChange}/>
<Button
variant="outlined"
color="primary"
onClick={handleLogin}>
          Login
</Button>
    </Stack>
);
```

6. Implement the change handler function for the `TextField` components, in order to save typed values to the states. You have to use the spread syntax because it ensures that you retain all the other properties of the `user` object that are not modified in this update:

```
const handleChange = (event: React.ChangeEvent<HTMLInputElement>) =>
  {   setUser({...user,
     [event.target.name] : event.target.value
  });
}
```

7. As shown in *Chapter 5*, *Securing Your Backend*, the login is done by calling the `/login` endpoint using the `POST` method and sending the `user` object inside the body. If authentication succeeds, we get a token in a response `Authorization` header. We will then save the token to session storage and set the `isAuthenticated` state value to `true`.

Session storage is similar to local storage, but it is cleared when a page session ends (when the page is closed). `localStorage` and `sessionStorage` are properties of the `Window` interface.

When the `isAuthenticated` state value is changed, the user interface is re-rendered:

```
const handleLogin = () => {
axios.post(import.meta.env.VITE_API_URL + "/login", user, {
headers: { 'Content-Type': 'application/json' }
  })
  .then(res => {     const jwtToken = res.headers.authorization;

    if    (jwtToken    !==    null)    {
sessionStorage.setItem("jwt", jwtToken);
setAuth(true);
    }
  })
  .catch(err => console.error(err));
}
```

8. We will implement some conditional rendering that renders the `Login` component if the `isAuthenticated` state is `false`, or the `Carlist` component if the `isAuthenticated` state is `true`. First, import the `Carlist` component into the `Login.tsx` file:

```
import Carlist from './Carlist';
```

Then, implement the following changes to the `return` statement:

```
if (isAuthenticated) {
return <Carlist />;
} else {
return(
    <Stack spacing={2} alignItems="center" mt={2} >


      <TextField
name="username"
label="Username"
onChange={handleChange} />
<TextField
type="password"
name="password"
label="Password"
onChange={handleChange}/>
<Button
variant="outlined"
color="primary"
onClick={handleLogin}>
          Login
</Button>
    </Stack>
  );
}
```

9. To show the login form, we have to render the `Login` component instead of the `Carlist` component in the `App.tsx` file. Import and render the `Login` component and remove the unused `Carlist` import:

```
// App.tsx import AppBar from '@mui/material/AppBar'; import
Toolbar from '@mui/material/Toolbar'; import Typography from
'@mui/material/Typography'; import Container from
'@mui/material/Container'; impor import Login from m
'@r './components/Login'; ıe';
                          import { QueryClient, QueryClientProvider
                        }
from '@tanstack/react  query';

const queryClient = new QueryClient();

function App() {   return
(
    <Container maxWidth="xl">



      <CssBaseline />
      <AppBar position="static">
<Toolbar>
          <Typography variant="h6">
            Carshop
          </Typography>
        </Toolbar>
</AppBar>
      <QueryClientProvider client={queryClient}>
        <Login />
      </QueryClientProvider>
    </Container>
  )
}

export default App;
```

Now, when your frontend and backend are running, your frontend should look like the following screenshot:

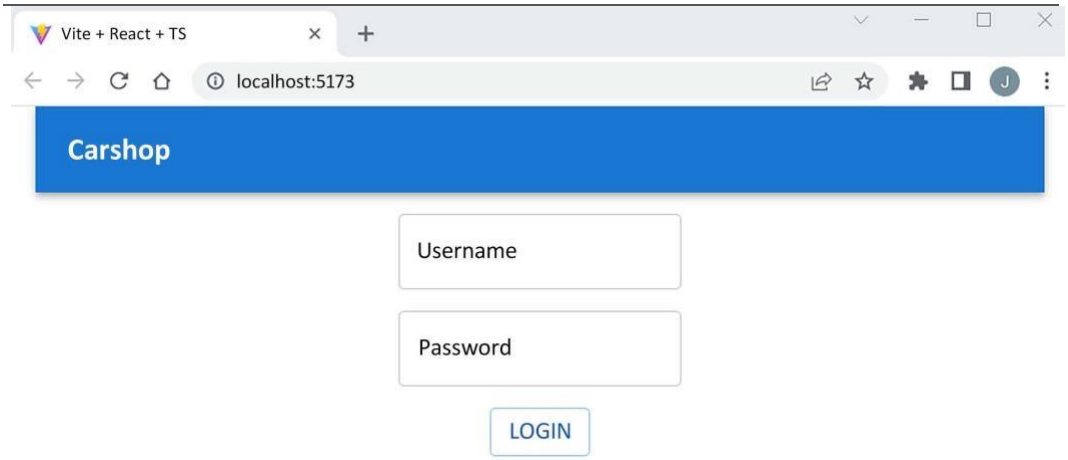

Figure 16.4: Login page

If you log in using the *user*/*user* or *admin*/*admin* credentials that we have inserted into the database, you should see the car list page. If you open the developer tools' **Application** tab, you can see that the token is now saved to session storage:

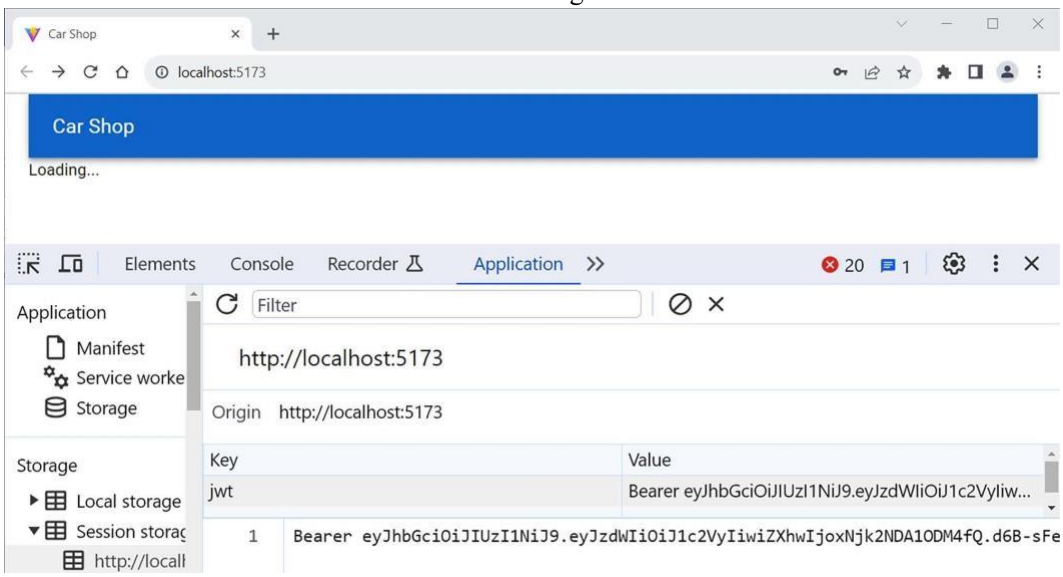

Figure 16.5: Session storage

Implementing REST API calls

At the end of the previous section, the car list is still loading and we can't fetch cars. This is the correct behavior because we haven't included the token in any requests yet. That is required for JWT authentication, which we will implement in the next phase:

1. Open the `carapi.ts` file in the VS Code editor view. To fetch the cars, we first have to read the token from session storage and then add the `Authorization` header with the token value to the `GET` request. You can see the source code for the `getCars` function here:

```
// carapi.ts export const getCars = async (): Promise<CarResponse[]>
=> {      const token = sessionStorage.getItem("jwt"); const response
= await axios.get(`${import.meta.env.VITE_API_URL}/
api/cars`,    {     headers: { 'Authorization' : token }
  });
return response.data._embedded.cars;
}
```

2. If you log in to your frontend, you should see the car list populated with cars from the database.
3. Check the request content from the developer tools; you can see that it contains the `Authorization` header with the token value:

Figure 16.6: Request headers

4. Modify the other CRUD functionalities in the same way so they work correctly. The source code for the `deleteCar` function appears as follows, after the modifications:

```
// carconst token = const deleteCar = async (link: string):
Prosessionstorage.getItem("jwt"); t
                    response = await axios.delete(link, {
                                headers: { 'Authorization': token }
  })   return response.data
}
```

The source code for the `addCar` and `editCar` functions appears as follows, after the modifications:

```
// carapi.ts export const addCar = async (car: Car):
Promise<CarResponse> => {   const token =
sessionStorage.getItem("jwt");
  const response = await axios.post(`${import.meta.env.VITE_API_
URL}/api/cars`, car, {     headers: {
      'Content-Type': 'application/json',
      'Authorization': token
    },
  });

  return response.data;
}

export const updateCar = async (carEntry: CarEntry):
Promise<CarResponse> => {   const token =
sessionStorage.getItem("jwt");  const response = await
axios.put(carEntry.url, carEntry.car, {     headers: {
      'Content-Type': 'application/json',
      'Authorization': token
    },
  });     return
response.data;
}
```

Refactoring duplicate code

Now, all the CRUD functionalities will work after you have logged in to the application. But, as you can see, we have quite a lot of duplicate code, such as the lines where we retrieve our token from session storage. We can do some refactoring to avoid repeating the same code and make our code easier to maintain:

1. First, we will create a function that retrieves the token from session storage and creates a configuration object for Axios requests that contains headers with the token. Axios provides the `AxiosRequestConfig` interface, which can be used to configure requests we send using Axios. We also set the `content-type` header value to `application/json`:

```
const getAxiosConfig = (): AxiosRequestConfig => {
const token = sessionStorage.getItem("jwt");

  return {
headers: {
      'Authorization': token,
      'Content-Type': 'application/json',
    },
  };
};
// carapi.ts import axios, { AxiosRequestConfig }
from 'axios'; import { CarResponse, Car, CarEntry }
from '../types';
```

2. Then, we can use the `getAxiosConfig()` function without retrieving a token in each function, by removing the configuration object and calling the `getAxiosConfig()` function instead, as shown in the following code:

```
// carapi.ts export const getCars = async (): Promise<CarResponse[]>
=> {   const response = await
axios.get(`${import.meta.env.VITE_API_URL}/
api/cars`, getAxiosConfig());   return response.data._embedded.cars;
}

export const deleteCar = async (link: string): Promise<CarResponse>
=>
{   const response = await axios.delete(link,
getAxiosConfig())   return response.data
}

export const addCar = async (car: Car): Promise<CarResponse> => {
const response = await axios.post(`${import.meta.env.VITE_API_
URL}/api/cars`, car, getAxiosConfig());

  return response.data;
}

export const updateCar = async (carEntry: CarEntry):
Promise<CarResponse> => {   const response = await
axios.put(carEntry.url, carEntry.car,
getAxiosConfig());

return response.data;
}
```

Axios also provides **interceptors** that can be used to intercept and modify requests and responses before they are handled by `then` or `catch`. You can read more about interceptors in the Axios documentation `https://axios-http.com/docs/interceptors`.

Displaying an error message

In this phase, we are going to implement an error message that is shown to a user if authentication fails. We will use the `Snackbar` MUI component to show the message:

1. Add the following import to the `Login.tsx` file:

```
import Snackbar from '@mui/material/Snackbar';
```

2. Add a new state called `open` to control the visibility of the `Snackbar`:

```
const [open, setOpen] = useState(false);
```

3. Add the `Snackbar` component to the `return` statement, inside the stack just under the `Button` component. The `Snackbar` component is used to show toast messages. The component is shown if the `open` prop value is `true`. The `autoHideDuration` defines the number of milliseconds to wait before the `onClose` function is called:

```
<Snackbar      open={open}      autoHideDuration={3000}
onClose={() => setOpen(false)}   message="Login failed:
Check your username and password" />
```

4. Open the `Snackbar` component if authentication fails by setting the `open` state value to `true`:

```
const login = () => {
axios.post(import.meta.env.VITE_API_URL + "/login", user, {
headers: { 'Content-Type': 'application/json' }
  })
  .then(res => {     const jwtToken =
res.headers.authorization;


    if (jwtToken !== null) {       sessionStorage.setItem("jwt",
jwtToken);       setAuth(true);
    }
  })
  .catch(() => setOpen(true));
}
```

5. If you now try to log in with the wrong credentials, you will see the following message in the bottom-left corner of the screen:

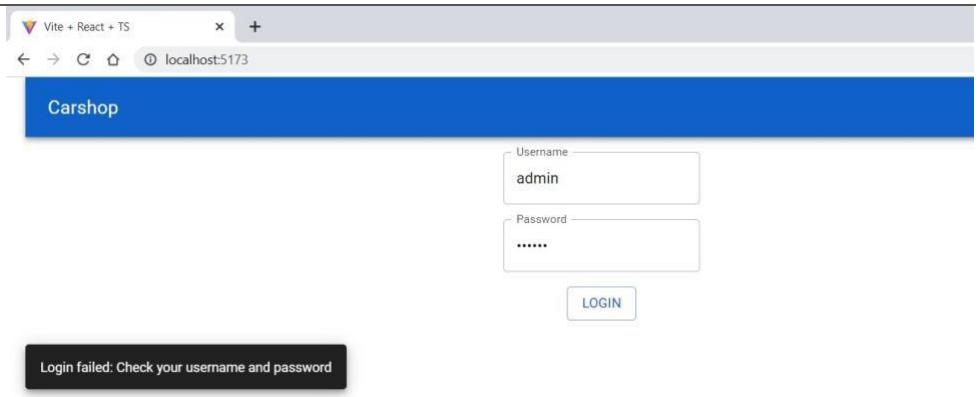

Figure 16.7: Login failed

Logging out

In this last section, we will implement the logout functionality in the `Login` component. The logout button is rendered on the car list page. The `Carlist` component is a child component of the `Login` component; therefore, we can pass the logout function to the car list using the props. Let's do this:

1. First, we create a `handleLogout()` function for the `Login` component, which updates the `isAuthenticated` state to `false` and clears the token from session storage:

```
// Login.tsx const handleLogout = () => {
setAuth(false);   sessionStorage.setItem("jwt",
"");
}
```

2. Next, we pass the `handleLogout` function to the `Carlist` component using the props, as shown in the highlighted code:

```
// Login.tsx if (isAuthenticated) {
return <Carlist    logOut={handleLogout}    />;
} else {
return(
...
```

3. We have to create a new type for the props that we receive in the `Carlist` component. The prop name is `logOut`, which is a function that takes no arguments, and we mark this prop as optional. Add the following type to the `Carlist` component and receive the `logOut` prop in the function arguments:

```
//Carlist.tsx type
CarlistProps =
{
logOut?:  ()  =>
void;
}

function Carlist(  ){ logOut }: CarlistProps{
const [open, setOpen] = useState(false);
...
```

4. Now, we can call the logout function and add the logout button. We use the Material UI `Stack` component to align the buttons so that the **NEW CAR** button is on the left and the **LOG OUT** button is on the right side of the screen:

```
// Carlist.tsx // Add the following imports import
Button from
'@mui/material/Button'; import Stack from
'@mui/material/Stack';

// Render the Stack and Button  if
(!isSuccess) {     return
<span>Loading...</span>
}
```

```
else if (error) {   return <span>Error when
fetching cars...</span>
} else {   return
(
    <>
```

```
        </Stack>
            <Stack
            direction="row"
      alignItems="center"
      justifyContent="space-   <AddCar
      between">
      />
              <Button
              onClick={logOut}>Log
              out</Button>
                     <DataGrid
rows={data}         columns={columns}
disableRowSelectionOnClick={true}
slots={{ toolbar: GridToolbar }}
getRowId={row => row._links.self.href} />
<Snackbar         open={open}
autoHideDuration={2000}         onClose={()
=> setOpen(false)}         message="Car
deleted" />     </>
  );
}
```

5. Now, if you log in to your frontend, you can see the **LOG OUT** button on the car list page, as shown in the following screenshot. When you click the button, the login page is rendered because the `isAuthenticated` state is set to `false` and the token is cleared from session storage:

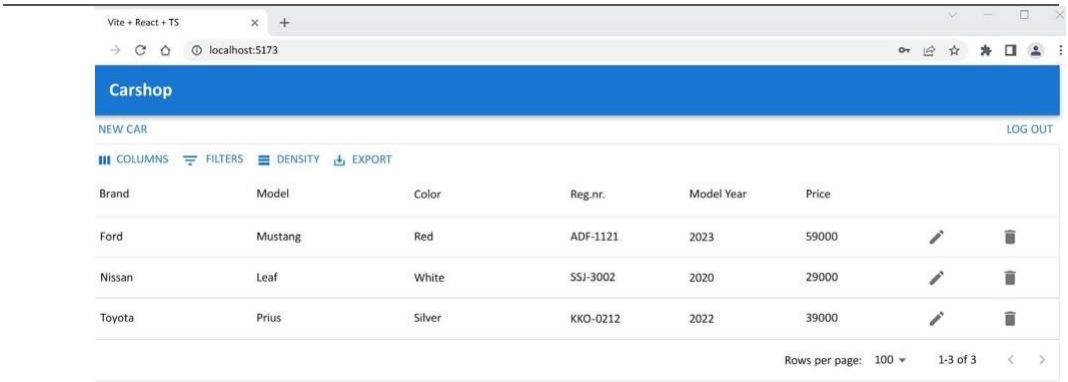

Figure 16.8: Log out

If you have a more complicated frontend with multiple pages, it would be wise to render the logout button in the app bar so that it is shown on each page. Then, you can use a state management technique to share a state with the whole component tree in your app. One solution would be to use the **React Context API** that we introduced in *Chapter 8*, *Getting Started with React*. In this scenario, you could use context to share the `isAuthenticated` state in your application's component tree.

As your application grows in complexity, managing state becomes crucial to ensuring that your components can access and update data efficiently. There are also other alternatives to the React Context API to manage states that you can study. The most common state management libraries are **React Redux** (`https://react-redux.js.org`) and MobX (`https://github.com/mobxjs/mobx`).

In the previous chapter, we created test cases for the `CarList` component, and at that point the app was unsecured. At this stage, our `CarList` component test cases will fail, and you should refactor them. To create a React test that simulates a login process and then tests whether data is fetched from a backend REST API, you can also use libraries like `axios-mock-adapter` (`https://github.com/ctimmerm/axios-mock-adapter`). Mocking Axios allows you to simulate the login process and data fetching without making actual network requests. We are not going into the details here, but we recommend you explore this further.

Now, we are ready with our car application.